

“学”：基于 Nexys A7 的 VGA 游戏集合机

从 VGA 显示、PS/2 输入到资源分析与时序收敛

VGA

PS2

Game Slot

LUT

WNS

这不是单个小游戏，而是一台 FPGA 游戏集合机

从用户角度，它有统一入口、统一输入和返回路径。

图示：user_flow

从用户角度，它有统一入口、统一输入和返回路径。

- ☐ 上电后先进入 VGA 菜单
- ☐ PS/2 键盘选择并启动游戏
- ☐ 游戏中按 Esc 返回菜单
- ☐ 四个槽位：TANK WAR / GAME TWO / GAME THREE / GAME FOUR

我的学习路线：从能显示，到能优化

本次汇报按学习路径讲，不按文件列表讲。

图示：learning_route

本次汇报按学习路径讲，不按文件列表讲。

- ☐ 先解释 VGA 实时像素扫描
- ☐ 再解释菜单渲染和 PS/2 输入事件化
- ☐ 然后说明 Game Slot API 的平台化意义
- ☐ 最后用资源和时序报告说明工程优化过程

主机 + 游戏槽位: game_console_top 如何组织系统

顶层提供公共服务，槽位只负责自身逻辑和画面。

图示: [architecture](#)

顶层提供公共服务，槽位只负责自身逻辑和画面。

- ☐ common 模块: VGA 同步、菜单渲染、PS/2 接收、菜单控制
- ☐ slot1 到 slot4 使用统一端口接入
- ☐ VGA、LED、七段管、蜂鸣器由顶层统一复用

VGA 的本质：每个像素周期输出一个颜色

FPGA 不是发送整张图片，而是扫描到哪里就实时给出哪里的 RGB。

图示：vga_scan

FPGA 不是发送整张图片，而是扫描到哪里就实时给出哪里的 RGB。

- ☐ `pixel\_x / pixel\_y` 表示当前扫描坐标
- ☐ `display\_active` 表示当前是否在 640x480 可见区
- ☐ `VGA\_R/G/B` 给出当前像素颜色
- ☐ `VGA\_HS / VGA\_VS` 让显示器识别行和帧同步

console_vga_sync: 用计数器生成像素坐标

VGA 控制器本质上是两个计数器加同步脉冲生成逻辑。

```
// src/common/console_vga_sync.v
localparam integer H_VISIBLE = 640;
localparam integer H_FRONT   = 16;
localparam integer H_SYNC    = 96;
localparam integer H_BACK    = 48;
localparam integer V_VISIBLE = 480;
localparam integer V_FRONT   = 10;
localparam integer V_SYNC    = 2;
localparam integer V_BACK    = 33;

reg [1:0] pix_div;
assign pixel_tick = (pix_div == 2'b11);
assign display_active = (h_count < H_VISIBLE) &&
                        (v_count < V_VISIBLE);
```

- ☐ 可见区为 640x480，扫描还包含 front/sync/back 消隐区
- ☐ `pix\_div` 每 4 个 100MHz 周期产生一次像素使能
- ☐ `h\_count/v\_count` 同步推进，得到当前像素坐标

单时钟域设计：100MHz 主时钟 + pixel_tick 使能

没有新建 25MHz 时钟，而是用 clock enable 保持单时钟域。

图示：single_clock

没有新建 25MHz 时钟，而是用 clock enable 保持单时钟域。

- ☐ 板卡主时钟是 100MHz
- ☐ `pixel\_tick` 作为 VGA 像素推进使能
- ☐ 所有主要逻辑仍由 `CLK100MHZ` 驱动
- ☐ 时序约束和跨模块协作更清晰

菜单不是图片，而是一组像素级判断

菜单渲染同样是实时像素逻辑，也是 LUT 压力来源之一。

```
// src/common/console_menu_renderer.v
if (!display_active)
    rgb = 12'h000;
else if (pixel [3:0])
    rgb = title_color;
else if (pixel [7:4])
    rgb = selected ? highlight_color : normal_text;
else if (pixel [11:8])
    rgb = border_color;
else
    rgb = background_color;
```

- ☐ `pixel\_x/y` 进入区域判断
- ☐ 标题、菜单项、帮助文字和边框都由坐标命中决定
- ☐ `cursor` 只决定哪一项高亮

菜单控制器决定“选谁”，渲染器决定“怎么画”

输入事件、菜单状态和像素渲染分层，顶层再做显示复用。

图示: `menu_control`

输入事件、菜单状态和像素渲染分层，顶层再做显示复用。

- ☐ ``console_ps2_rx`` 输出 `byte` 事件
- ☐ ``console_menu_controller`` 更新 ``cursor / game_sel / menu_active``
- ☐ ``console_menu_renderer`` 根据 ``cursor`` 和坐标输出菜单 RGB
- ☐ 顶层根据 ``menu_active`` 选择菜单或当前游戏画面

PS/2 输入不是按键电平，而是串行协议

键盘先发送扫描码，FPGA 必须先协议解析再事件化。

图示: `ps2_frame`

键盘先发送扫描码，FPGA 必须先协议解析再事件化。

- ☐ 一帧包含 start、8 位 scan code、parity、stop
- ☐ 数据位 LSB first
- ☐ 在 PS/2 时钟下降沿采样
- ☐ 底层只输出 ``byte_ready + byte_data``

console_ps2_rx: 下降沿采样并组装扫描码

这个模块只做协议层接收，不解释具体按键含义。

```
// src/common/console_ps2_rx.v
assign ps2_clk_fall_next = ps2_clk_q & ~ps2_clk_next;
assign ps2_parity_ok = (^shift_data) ^ parity_bit;

if (ps2_clk_fall_next) begin
  case (bit_count)
    4'd0: if (ps2_data_next == 1'b0)
      bit_count <= 4'd1;
    4'd1: begin shift_data[0] <= ps2_data_next;
      bit_count <= 4'd2; end
    // ... shift_data[1] ~ shift_data[7], parity
    4'd10: if (ps2_data_next == 1'b1 &&
      ps2_parity_ok) begin
      byte_data <= shift_data;
      byte_ready <= 1'b1;
    end
  endcase
end
```

end

- ☐ PS/2 时钟和数据先同步并做历史滤波
- ☐ `bit\_count` 记录当前接收到帧的哪一位
- ☐ 停止位为 1 且奇校验通过时产生 `byte\_ready`

扫描码事件化：W/S、Enter、Esc 如何控制菜单

底层 byte 被转换成上移、下移、启动、返回这些系统事件。

```
// src/common/console_menu_controller.v
localparam [7:0] SCAN_F0    = 8'hF0;
localparam [7:0] SCAN_E0    = 8'hE0;
localparam [7:0] SCAN_W     = 8'h1D;
localparam [7:0] SCAN_S     = 8'h1B;
localparam [7:0] SCAN_SPACE = 8'h29;
localparam [7:0] SCAN_ENTER = 8'h5A;
localparam [7:0] SCAN_ESC   = 8'h76;
localparam [7:0] SCAN_UP    = 8'h75;
localparam [7:0] SCAN_DOWN  = 8'h72;

assign is_up_key = (byte_data == SCAN_W) ||
                  (byte_data == SCAN_UP);
assign is_start_key = (byte_data == SCAN_SPACE) ||
                     (byte_data == SCAN_ENTER);
```

- ☐ `F0` 表示 key release 前缀，下一码不触发动作
- ☐ `E0` 记录扩展键前缀，方向键扫描码仍由上层识别
- ☐ 菜单内启动游戏；游戏中 Esc 返回菜单

统一槽位 API：从“多个游戏”到“游戏平台”

每个游戏像卡带一样接入，顶层只依赖统一端口。

图示：game_slot_api

每个游戏像卡带一样接入，顶层只依赖统一端口。

- ☐ 公共输入：`clk / reset / selected`
- ☐ 显示输入：`frame\_tick / pixel\_tick / display\_active / pixel\_x/y`
- ☐ 输入事件：`ps2\_byte\_ready / ps2\_byte\_data`、按钮和开关
- ☐ 统一输出：VGA RGB、LED、七段管、蜂鸣器

输出复用：菜单激活显示菜单，否则显示当前游戏

真实 VGA 端口只有一组，所有菜单和游戏画面都必须由顶层仲裁。

```
// src/game_console_top.v
assign active_slot_vga_r = slot1_selected ? slot1_vga_r :
                             slot2_selected ? slot2_vga_r :
                             slot3_selected ? slot3_vga_r :
                             slot4_vga_r;

always @(*) begin
    if (menu_active) begin
        VGA_R = menu_vga_r_q;
        VGA_G = menu_vga_g_q;
        VGA_B = menu_vga_b_q;
    end else begin
        VGA_R = active_slot_vga_r;
        VGA_G = active_slot_vga_g;
        VGA_B = active_slot_vga_b;
    end
end
```

- ☐ slot1 到 slot4 先复用为 `active\_slot\_\*`
- ☐ 菜单态覆盖 VGA、LED、七段管和蜂鸣器输出
- ☐ HS/VS 始终来自公共 `console\_vga\_sync`

资源报告告诉我：问题不是寄存器太多，而是组合逻辑太重

LUT 约 48%，FF 约 3%，说明组合路径和像素渲染是主要压力。

图示：resource_chart

LUT 约 48%，FF 约 3%，说明组合路径和像素渲染是主要压力。

- ☐ LUT: 30146 / 63400, 约 48%
- ☐ FF: 4371 / 126800, 约 3%
- ☐ 菜单、文字、地图、sprite、tile 和对象命中检测都会消耗 LUT
- ☐ 优化方向是适当增加寄存器和 staging，而不是继续减少 FF

每个像素都要回答一个问题：这里该显示什么？

VGA 游戏的复杂度来自每个像素周期的坐标判断和图层优先级。

图示: `pixel_decision`

VGA 游戏的复杂度来自每个像素周期的坐标判断和图层优先级。

- ☐ 是否在可见区
- ☐ 是否命中背景、地图 tile、玩家、敌人、子弹、道具
- ☐ 是否命中文字或 UI
- ☐ 最终按图层优先级选择 RGB

代码能综合，不代表电路能按 100MHz 跑

WNS 为负说明至少有一条路径在一个周期内来不及。

图示: `timing_concept`

WNS 为负说明至少有一条路径在一个周期内来不及。

- ☐ 100MHz 时钟周期为 10ns
- ☐ 寄存器 A 到寄存器 B 中间的组合逻辑必须在周期内稳定
- ☐ WNS: 最差路径时序余量
- ☐ TNS: 所有失败路径负 slack 总和

最终优化结果：从负 slack 到时序收敛

先暴露真实基线，再通过 RTL staging/pipeline 把 WNS 拉正。

图示: `timing_results`

先暴露真实基线，再通过 RTL staging/pipeline 把 WNS 拉正。

- ☐ 真实基线: WNS=-0.445ns, TNS=-18.104ns, 失败端点 94
- ☐ 最终 post-route: WNS=+0.174ns, TNS=0.000ns, 失败端点 0
- ☐ 只保留窄范围 VGA pixel multicycle, 不覆盖游戏核心状态
- ☐ 最终成功生成 bitstream

优化思想：把一拍大计算拆成多拍小计算

用户可见行为不变，但硬件每拍需要完成的逻辑更少。

图示: `optimization_cases`

用户可见行为不变，但硬件每拍需要完成的逻辑更少。

- ☐ Slot2: ghost 迭代、lock row-by-row、video staging
- ☐ Slot3: ``try_x` / `try_y` 寄存，下一拍判断 walkable`
- ☐ Slot4: `tile probe` $\rightarrow$ `eval` $\rightarrow$ `apply`
- ☐ 约束: 4-cycle VGA multicycle 只用于 `pixel_tick` 视频路径

从功能实现到工程实现

最终收获是把显示、输入、架构、资源和时序统一起来。

图示: summary

最终收获是把显示、输入、架构、资源和时序统一起来。

- ☐ VGA: 理解实时像素扫描
- ☐ 菜单: 理解坐标判断和图层优先级
- ☐ PS/2: 理解外设协议解析和事件化
- ☐ Game Slot API: 理解平台化系统架构
- ☐ LUT/WNS: 理解从 Verilog 功能到可实现硬件的差距